

Door Terminal Hardware Handover

Complete pinouts, wiring diagrams and repair reference
for the GYM-001 biometric entrance terminal

This document is written for a technician who has never seen this build before and has to repair or rebuild it. Every pin number, voltage and wire colour here is taken from the firmware that is actually running on the device (`firmware/config.py`) rather than from a design note, so what is drawn is what the board expects.

DEVICE CODE GYM-001 — Main Entrance	CONTROLLER ESP32-S3-DevKitC-1 N16R8
FIRMWARE MicroPython 1.23+ · NOVA v1.0.0	FINGERPRINT SENSOR R307 (1000 templates)
DOCUMENT REVISION Rev B — 10 September 2026	SUPERSEDES HARDWARE_WIRING.html (Rev A)

READ THIS BEFORE USING ANY OLDER DRAWING

An earlier document, `docs/HARDWARE_WIRING.html` and the PDF exported from it, is **obsolete and its pin map is wrong for this build**. It was drawn around the R503 sensor and a different GPIO allocation — it shows the modem on GPIO4/5, the buzzer on GPIO42 and the relay on GPIO40, none of which match the running firmware. Wiring a board to it will not work. Use this document, and delete or clearly mark the old one.

CREDENTIALS ARE DELIBERATELY NOT IN THIS DOCUMENT

The device key, the gym's Wi-Fi password and the Supabase keys live in `firmware/config.py` on the device and are not reproduced here, because this document is meant to be printed and handed to a repairer. If a replacement controller is fitted, issue it a fresh key with `node scripts/provision-device.mjs GYM-001 "Main Entrance"` — that also revokes the old one immediately. See §11.

00 Contents

01	System overview	What the terminal is and how the parts relate
02	Bill of materials	Every part, with the spares worth stocking
03	Power distribution	12 V in, the buck, the star ground, who runs on what
04	Master GPIO map	One table: every pin the firmware claims
05	Complete wiring diagram	The whole terminal on one page
06	Module connection tables	Per-module pinouts and wire colours
07	Lock drive stage	Relay, solenoid, flyback and fusing
08	Reserved and forbidden pins	What will brick the board if reused
09	Bring-up procedure	Powering a repaired board in a safe order
10	Fault-finding	Symptom → cause → fix, and the diagnostic scripts
11	Replacing a part	Controller, sensor, display, lock, modem
12	Safety and compliance	Egress, mains, and what must never be firmware

How to use this document

If the door is dead, start at **\$10 Fault-finding** — it is arranged by symptom. If you are rebuilding or rewiring, work from **\$5** (the diagram), **\$6** (the connection tables) and then **\$9** (the bring-up order, which matters). **\$4** is the single-page pin map to keep beside the bench.

THE ONE SOURCE OF TRUTH

If this document and the device ever disagree, `firmware/config.py` wins — it is what the code reads at boot. Every pin in \$4 is copied from it. If someone changes a pin in that file, this document is out of date until it is regenerated.

01 System overview

A member puts a finger on the sensor. The terminal matches it locally, asks the server whether that member is allowed in, shows the verdict on the screen and pulses the relay that releases the door. If the network is down it decides from a cached list and queues the event for later.

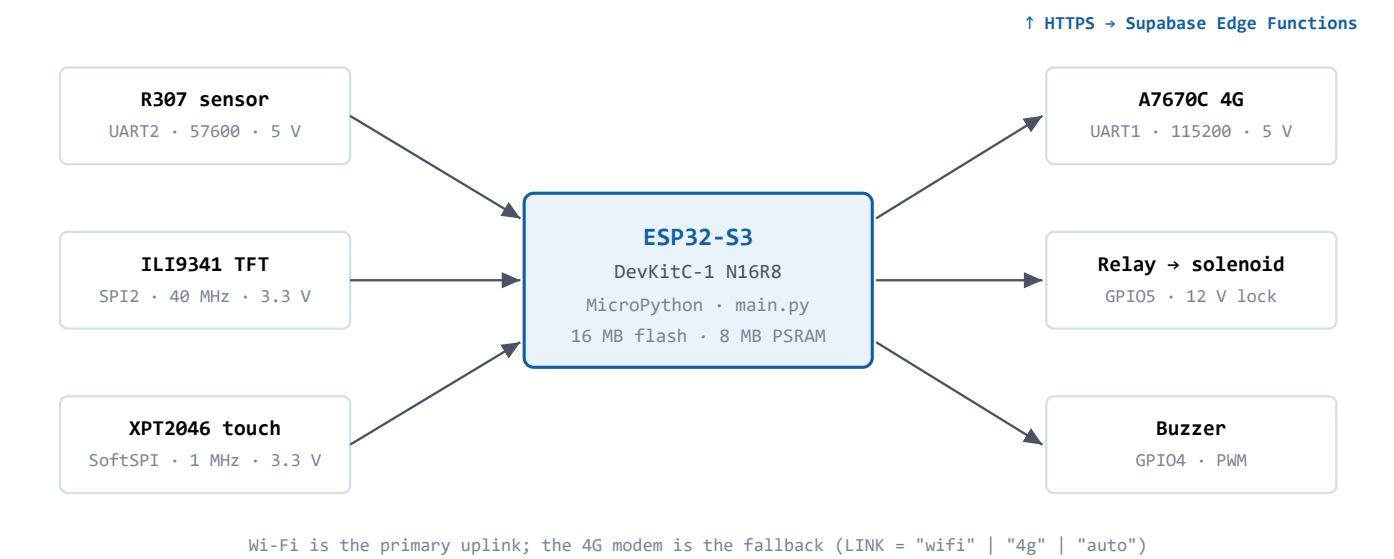


FIG. 1 – SIGNAL OVERVIEW

What talks to what

The terminal never reaches the database directly and never holds an administrative key. It calls a small set of HTTPS endpoints, authenticating with a per-device key whose hash is the only copy stored on the server. The web dashboard is a separate client of the same backend — the two never talk to each other. When staff press *Enroll Fingerprint* on the dashboard, that writes a row the device polls for.

INTERFACES IN USE

BUS	PERIPHERAL	SPEED	NOTES
UART0	USB console	115200	REPL / Thonny. Do not reuse.
UART1	A7670C modem	115200	Autobauds, settles at 115200.
UART2	R307 sensor	57600	8N1. ZFM protocol.
SPI2 (hardware)	ILI9341 panel	40 MHz	Panel owns this peripheral exclusively.
SoftSPI (bit-banged)	XPT2046 touch	1 MHz	Separate on purpose — see note below.

WHY TOUCH IS ON ITS OWN BUS

The ILI9341 runs at 40 MHz and the XPT2046 tops out near 2 MHz. Sharing one SPI peripheral would mean re-clocking the bus on every touch poll, which shows as visible tearing on the screen. Worse, opening hardware SPI(2) a second time for touch silently re-points the peripheral at the touch pins and freezes the display with no error. The touch controller is therefore bit-banged in software on its own five pins. Do not "optimise" this by merging the buses.

02 Bill of materials

The modules as fitted, plus the small parts that are not modules but matter. Quantities are per terminal.

MODULES

PART	QTY	SUPPLY	ROLE AND NOTES
ESP32-S3-DevKitC-1 N16R8	1	5 V / USB	Main controller. 16 MB flash, 8 MB octal PSRAM. The SPIRAM MicroPython build is required.
R307 fingerprint module	1	5 V	Optical, 1000 templates. Replaced the R503 for capacity. Will not start below 4.2 V.
2.4" ILI9341 TFT MD0671, 240×320	1	3.3 V	SPI display with XPT2046 resistive touch. Must be the touch variant — see §11.
SIMCom A7670C 4G LTE FS-MCore V1.2	1	5 V	Fallback uplink. Needs a nano SIM with a data plan and its antenna fitted.
HW-688 buck converter 12/24 V → 5 V, 6 A	1	12 V in	The 5 V rail for everything. Set its output before connecting any load.
1-channel relay module 5 V, optocoupler	1	5 V	Switches the lock. Optocoupler isolation is the reason the lock's noise does not reach the SPI bus.
12 V solenoid door lock	1	12 V	Fail-secure assumed — locked with no power. Check yours before powering; see §7.
Passive buzzer	1	3.3 V	Driven by PWM on GPIO4 for four distinct cues.
KY-025 reed switch	1	—	Supplied but NOT fitted. <code>PIN_REED = None</code> . Nothing in the firmware waits on a door contact. Keep as a spare or fit per §11.

Small parts — not optional

DISCRETES AND HARDWARE

PART	QTY	WHERE IT GOES AND WHY
1N4007 diode	1	Across the solenoid coil, cathode to the +12 V side, mounted at the lock . Without it the inductive kick arcs the relay contacts and radiates enough noise to corrupt the SPI display.
470 µF electrolytic	1	At the modem's VIN terminal. The A7670C peaks at 2.3 A on transmit.
100 µF electrolytic	1	At the R307's power connector.
100 nF ceramic	2	One beside each electrolytic above. The electrolytic handles the millisecond sag; the ceramic handles the fast edge.
3 A fuse + holder	1	In the +12 V feed, before anything else.
SS34 Schottky	1	<i>Optional but recommended</i> — in the buck's 5 V output line, so USB VBUS and the buck cannot fight during flashing. See §9.
12 V PSU, ≥ 2 A	1	Sized for the lock's inrush plus the modem's peaks, not their averages.

SPARES WORTH KEEPING ON SITE

A spare **R307**, a spare **relay module** and a spare **3 A fuse** cover the three things most likely to fail in service: the sensor is what everyone touches, relay contacts wear out from switching an inductive load, and a fuse is what a lock fault takes out first. A spare ESP32-S3 is worth having but needs re-provisioning (\$11) rather than a straight swap.

03 Power distribution

One 12 V supply comes in. It feeds the lock directly and a buck converter that makes the 5 V rail. The ESP32's own regulator makes 3.3 V for the logic-level parts. Getting this tree wrong is the single most common cause of faults that look like software.

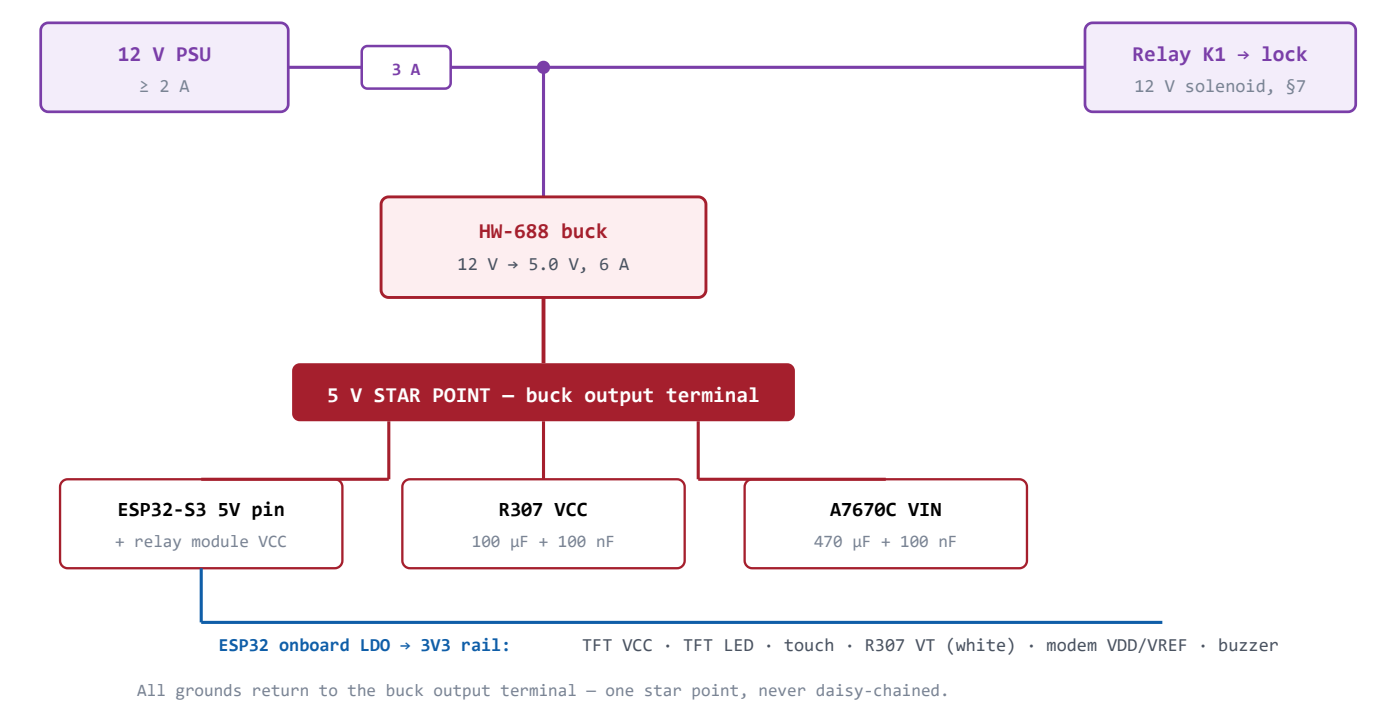


FIG. 2 – POWER TREE

WHO RUNS ON WHICH RAIL

LOAD	RAIL	DRAW	RULE
Solenoid lock	12 V	~500 mA	Through the relay contacts only. Never near logic.
ESP32-S3 board	5 V	~250 mA	Into the 5V header pin, from the star point.
R307 sensor	5 V	~120 mA	Direct to the buck terminal, not the ESP32's 5V pin.
A7670C modem	5 V	20 mA idle, 2.3 A peak	Direct to the buck terminal. The peak is the whole reason for the star point.
Relay module coil	5 V	~70 mA	VCC from 5 V; the input pin is 3.3 V logic through the optocoupler.
ILI9341 + touch	3.3 V	~90 mA	From the ESP32's 3V3 pin. Never 5 V — no level shifters on this panel.
R307 VT (white)	3.3 V	negligible	Only if the WAKEUP pin is used. See §6.
Modem VDD/VREF	3.3 V	negligible	The level-shifter reference. Nothing crosses the UART without it.

SET THE BUCK OUTPUT FIRST, WITH NOTHING CONNECTED

Some HW-688 modules ship with the trimmer set near their input voltage. Wire 12 V in with **nothing on the output**, meter the output terminals, adjust to 5.0 V, power down, then connect the loads. Connecting the ESP32 to an unverified buck output is a very short experiment.

WHY THE MODEM AND SENSOR BYPASS THE ESP32'S 5V PIN

The A7670C draws 2.3 A peaks during network registration and every TLS handshake, even though it idles near 20 mA. Fed through the ESP32's 5V pin and its traces, those peaks brown the board out mid-attach. The symptom is maddening and looks nothing like a power fault: *a modem that answers AT perfectly and then never registers*. Both high-current loads go to the buck's own terminal, and the 470 μ F sits right at the modem connector — for those 1–2 ms it is the entire supply.

GROUNDING

Every ground returns to the buck's output terminal. Do not daisy-chain the modem's ground through the ESP32's header — the modem's return current then flows through the controller's ground plane, and a shared-ground offset during a transmit burst is enough to corrupt UART framing on a completely unrelated peripheral.

04 Master GPIO map

Every pin the firmware claims, in pin order. Taken directly from `firmware/config.py`. Directions are from the ESP32's point of view.

ESP32-S3 PIN ALLOCATION — AS RUNNING

GPI	SIGNAL	DIR	BUS	GOES TO	NOTES
GPI04	BUZZER	out	PWM/LEDC	Buzzer +	Passive buzzer, four cues. Failures are swallowed in code so a dead buzzer never takes the door down.
GPI05	DOOR_RELAY	out	GPI	Relay IN	Active HIGH. Pulsed for 2000 ms on a granted scan.
GPI07	MODEM_RX	in	UART1	A7670C TX	Receives from the modem.
GPI08	TFT_RST	out	GPI	TFT RESET	Idles high.
GPI09	TFT_DC	out	GPI	TFT DC / RS	Data / command select.
GPI010	TFT_CS	out	SPI2	TFT CS	Panel chip select, idles high.
GPI011	TFT_MOSI	out	SPI2	TFT SDI / MOSI	40 MHz.
GPI012	TFT_SCK	out	SPI2	TFT SCK	40 MHz.
GPI013	TFT_MISO	in	SPI2	TFT SDO / MISO	Required for the controller ID read in <code>diag_display.py</code> .
GPI014	TCH_CLK	out	SoftSPI	T_CLK	Bit-banged, 1 MHz.
GPI015	TCH_CS	out	SoftSPI	T_CS	Idles high.
GPI016	FP_RX	in	UART2	R307 TXD (yellow)	Receives from the sensor. 57600 baud.
GPI017	FP_TX	out	UART2	R307 RXD (green)	Transmits to the sensor. 3.3 V direct, no level shifter.
GPI018	MODEM_TX	out	UART1	A7670C RX	Transmits to the modem.
GPI021	TCH_MOSI	out	SoftSPI	T_DIN	
GPI038	TCH_IRQ	in	GPI	T_IRQ	Pen-down. Configured <code>PULL_UP</code> , active low — no external resistor.
GPI039	TCH_MISO	in	SoftSPI	T_DO	Moved from GPI047 — that pad is damaged on this unit. Any free GPI works because the bus is bit-banged.
GPI040	MODEM_PWRKEY	out	GPI	A7670C PWK	Open-drain , idles high, pulsed low to toggle power. Never pull it up to 3.3 V.

Configured as unused

PRESENT IN THE FIRMWARE BUT SET TO NONE

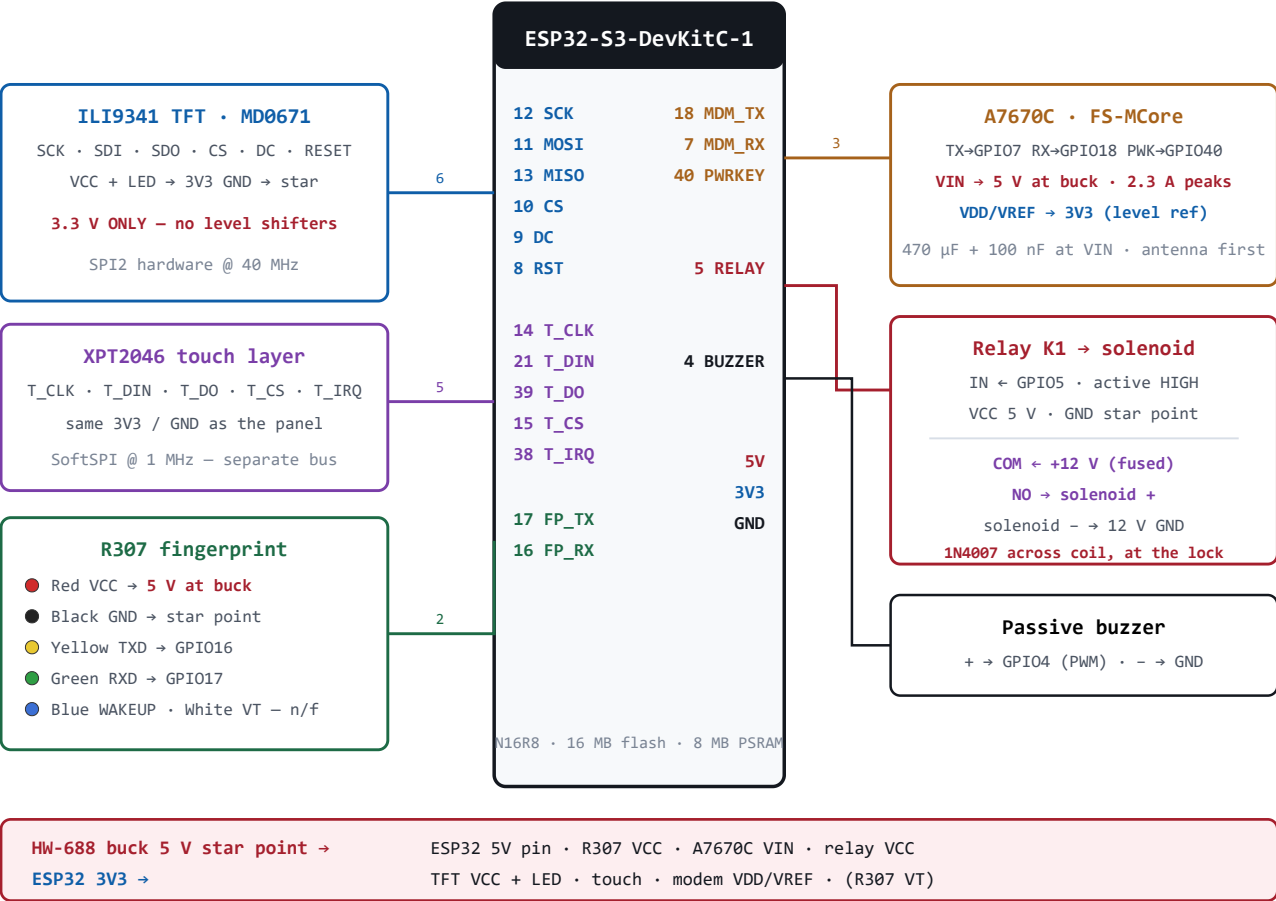
SETTING	SUGGESTED PIN	WHY IT IS OFF
PIN_FP_WAKEUP	GPI041	R307 finger-present output (blue wire). Not fitted, so the firmware polls the sensor instead. Fitting it improves responsiveness — see §6.
PIN_REED	—	No door contact fitted. A granted scan pulses the relay for a fixed time; nothing waits on a switch.
PIN_MODEM_NET	GPI041	Modem network-status LED. Informational only.

READING THE UART ROWS CORRECTLY

UART crossings are where rebuilds go wrong. The rule is that a *TX* always goes to an *RX*. On this board that means GPI017 → R307 RXD , GPI016 ← R307 TXD , GPI018 → modem RX , GPI07 ← modem TX . If a UART device is silent, swapping these two wires is the first thing to try — it costs nothing and it is the most common single fault.

05 Complete wiring diagram

The whole terminal on one page. Signal wires are drawn to the ESP32; power is shown by rail colour rather than by routing every line, because the routing rule is simply "back to the star point".



Numbers on the signal bundles are wire counts. All GND returns to the buck output terminal.

The KY-025 reed switch is supplied but not fitted, and no exit button is wired in this build.

FIG. 3 – COMPLETE INTERCONNECT

06 Module connection tables

One table per module, in the order you would wire them. Wire colours are the common convention — **measure before trusting them**, because suppliers vary.

6.1 · R307 fingerprint sensor

WIRE	FUNCTION	CONNECTS TO	NOTE
● Red	VCC	5 V at the HW-688 terminal	Not 3.3 V, not the ESP32's 5V pin. The R307 will not start below 4.2 V.
● Black	GND	Buck terminal (star point)	
● Yellow	TXD	<code>GPIO16</code>	Sensor → ESP32. Idles high at ~3.3 V.
● Green	RXD	<code>GPIO17</code>	ESP32 → sensor. Accepts 3.3 V; no level shifter needed.
● Blue	WAKEUP	<i>not fitted</i> (would be <code>GPIO41</code>)	Finger-present output. See the note below.
○ White	VT	<i>not fitted</i> (would be 3.3 V)	Powers the touch circuit that drives WAKEUP. Required <i>if and only if</i> blue is used.

Bus: UART2, 57600 baud, 8N1. **Decoupling:** 100 µF + 100 nF at the connector. **Capacity:** 1000 templates, but the firmware always asks the chip via `ReadSysPara` rather than trusting the label.

IF YOU FIT THE WAKEUP WIRE

Without it, the idle loop calls `get_image()` over and over and the sensor is *blind between calls* — a finger arriving in the gap is simply not seen, which members experience as "it only works sometimes". The blue capture light strobing is that polling made visible; the R307 lights it while imaging, not as a standby indicator.

To fit it: blue → `GPIO41`, **white** → **3.3 V**, then set `PIN_FP_WAKEUP = 41`. The white wire is not optional — WAKEUP is the output of a touch circuit that VT powers, and unpowered it never asserts, which is indistinguishable from a broken wire. Measure the polarity before trusting `FP_WAKEUP_ACTIVE_HIGH = True`; backwards means the door images constantly and never when you want it to.

ON A REPLACEMENT SENSOR, CHECK THE CAPACITY BEFORE ENROLLING ANYONE

These modules ship in 200- and 1000-template variants that sellers label interchangeably. Run `import selftest` and confirm it prints 1000. The alternative to one command now is discovering it at member 201.

6.2 · ILI9341 TFT display (MD0671)

PANEL

PANEL PIN	ESP32-S3
VCC	3V3
GND	GND (star)
CS	<code>GPIO10</code>
RESET	<code>GPIO8</code>

TOUCH LAYER

TOUCH PIN	ESP32-S3
T_CLK	<code>GPIO14</code>
T_CS	<code>GPIO15</code>
T_DIN	<code>GPIO21</code>

PANEL PIN	ESP32-S3
DC / RS	<code>GPIO9</code>
SDI / MOSI	<code>GPIO11</code>
SCK	<code>GPIO12</code>
LED	<code>3V3</code>
SDO / MISO	<code>GPIO13</code>

TOUCH PIN	ESP32-S3
T_DO	<code>GPIO39</code>
T_IRQ	<code>GPIO38</code>

Resolution: 240×320. **Panel bus:** hardware SPI2 at 40 MHz. **Touch bus:** SoftSPI at 1 MHz.

3.3 V ONLY
Run the whole panel at 3.3 V, including LED. These modules have no level shifters and 5 V on the logic pins is what kills them.

T_DO IS ON GPIO39, NOT GPIO47
The GPIO47 pad is physically damaged on this unit, so touch MISO was moved. Because the touch bus is bit-banged in software, any free GPIO works — if you fit a replacement controller with an intact GPIO47 you may move it back, but you must then change `PIN_TCH_MISO` in `config.py` to match.

IF THE REPLACEMENT PANEL HAS NO T_ PINS
The MD0671 ships in touch and non-touch versions. If the header has only the nine panel pins, the glass has no touch layer and no firmware can add one — you need the XPT2046 variant. Everything else runs unchanged on a non-touch panel; you lose the three on-screen buttons and enrollment then starts from the dashboard only.

6.3 · A7670C 4G modem (FS-MCore V1.2)

MODEM PIN	FUNCTION	CONNECTS TO	NOTE
VIN	Supply	5 V at buck terminal	5–16 V rated. 2.3 A transmit peaks — 470 µF + 100 nF at the connector.
GND	Ground	Buck terminal (star)	Must share ground with the ESP32.
VDD	UART level reference	3V3	The onboard level shifter is referenced to this. Without it nothing crosses in either direction.
TX	Modem transmit	<code>GPIO7</code>	
RX	Modem receive	<code>GPIO18</code>	
PWK	Power key	<code>GPIO40</code>	Open-drain, like a button to ground. Pulsed low to toggle power. Never pull up to 3.3 V.
NET	Network status LED	<i>unconnected</i>	<code>PIN_MODEM_NET = None</code> .
PEN	Power enable	<i>unconnected</i>	

Bus: UART1, 115200 baud. **SIM:** nano, contacts toward the board, **data plan required** — a voice-only SIM registers happily and then never gets an IP. **Fit the antenna before powering up.**

KNOWN-GOOD REFERENCE

This link has been verified end to end on a Dialog SIM: -71 dBm, registered on 41302 , APN dialogbb , a CGNAT address from AT+CGPADDR , and HTTP/1.1 200 OK from the heartbeat endpoint over LTE and TLS. If a replacement modem reaches those same numbers, the hardware is good and any remaining fault is configuration.

TWO AT SETTINGS THAT ARE NOT DEFAULTS, AND COST AN EVENING EACH

AT+CSSLCFG="enableSNI",0,1 — the backend sits behind Cloudflare, where one IP serves thousands of certificates. Without the hostname in the handshake the server cannot pick one. SIMCom ships this **off**.

AT+CSSLCFG="ignorelocaltime",0,1 — the modem has no battery-backed clock and reads 1970 until the network tells it otherwise, so every certificate looks not-yet-valid.

These are applied by the firmware. They are recorded here because the same fault appears three different ways and never once says "SNI".

6.4 · Relay module

RELAY PIN	CONNECTS TO	NOTE
VCC	5 V (star point)	Coil supply.
GND	GND (star point)	
IN	GPI05	Active HIGH (RELAY_ACTIVE_HIGH = True). 3.3 V drives the optocoupler fine.
COM	+12 V , after the fuse	Contact side — isolated from all logic.
NO	Solenoid +	Normally open: no power, no current, door stays locked.
NC	<i>unused</i>	

6.5 · Buzzer

Passive buzzer, + to **GPI04** , - to GND. Driven by PWM. If you fit an active buzzer instead it will sound at its own fixed pitch and the four distinct cues collapse into one tone — `diag_buzzer.py` sweeps a range of frequencies specifically to tell the two types apart.

6.6 · KY-025 reed switch — not fitted

Supplied in the parts list but deliberately not wired. `PIN_REED = None` , and the firmware pulses the relay for a fixed 2000 ms rather than waiting on a door contact. If you want door-ajar detection, wire D0 to a free input, set `PIN_REED` , and be aware the current firmware reads it but does not gate the unlock on it.

07 Lock drive stage

The only part of this build that switches real current. It is also the part most likely to damage something else when it is wrong.

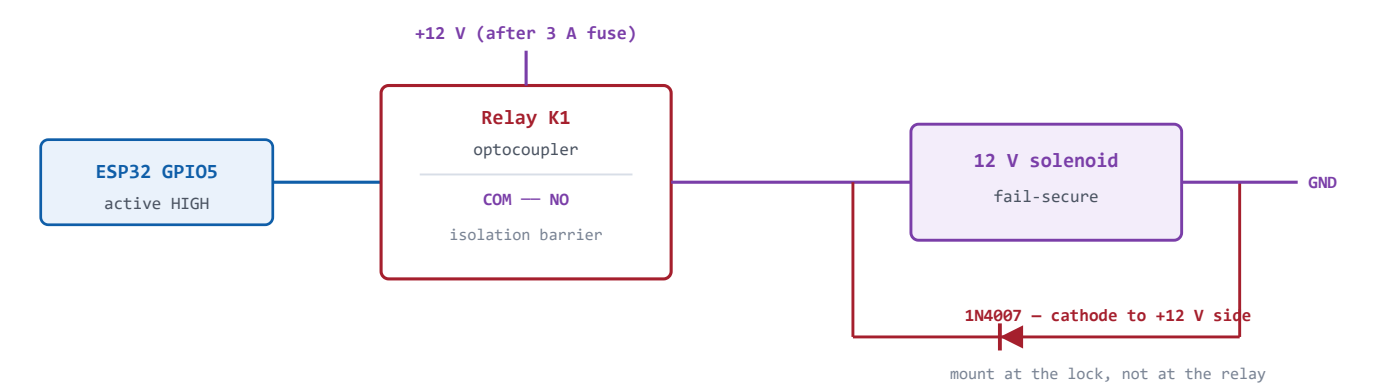


FIG. 4 – LOCK DRIVE STAGE

THE FLYBACK DIODE IS NOT OPTIONAL

Switching an inductive 12 V load with no diode arcs the relay contacts and radiates enough noise to corrupt the SPI display. A 1N4007 costs almost nothing and is the difference between a relay that lasts years and one that welds shut — welded closed means a door that never locks. **Mount it at the lock, not at the relay**, so the loop the current circulates in is as small as possible.

CHECK WHICH KIND OF LOCK YOU HAVE BEFORE POWERING UP

A **fail-secure** solenoid stays locked with no power and only draws current while the relay is closed. That is what is drawn above and the right default for a gym. A **fail-safe** lock is the opposite: energised continuously to stay unlocked, and it drops open on power loss. Fitting a fail-safe lock to this wiring would keep 12 V through the relay contacts 24/7. If yours is fail-safe you must invert the relay logic (`RELAY_ACTIVE_HIGH = False`) and size the supply for continuous draw — and re-read §12 first, because the egress behaviour changes completely.

TIMING AND BEHAVIOUR

SETTING	VALUE	MEANING
DOOR_UNLOCK_MS	2000	How long the relay stays closed after a granted scan. Long enough to push the door open before it re-locks.
RELAY_ACTIVE_HIGH	True	GPIO5 high = energised = unlocked.
PIN_DOOR_RELAY	5	Set to <code>None</code> to run display-only on the bench with no lock attached.

Most 12 V solenoids are rated for **intermittent duty** and will overheat if held energised. Do not raise `DOOR_UNLOCK_MS` far beyond a few seconds without checking the lock's datasheet.

Bench-testing the lock stage

On the device, with the solenoid *disconnected* first so you are listening for the relay click alone:

```
>>> import test_relay
```

Then connect the lock and repeat. Separating those two steps tells a dead relay apart from a dead lock in about thirty seconds.

08 Reserved and forbidden pins

If you need a spare GPIO — for the reed switch, an exit button, or a status LED — take it from the free list. Taking it from the forbidden list can make the board unbootable.

DO NOT REUSE ON THE N16R8

PINS	WHAT THEY ARE	CONSEQUENCE OF REUSING
GPIO26–32	SPI flash	Bricks the boot. Not brought out for a reason.
GPIO33–37	Octal PSRAM (R8)	Bricks the boot on this 8 MB octal-PSRAM variant. Free on non-PSRAM S3 boards, which is why bad advice about them circulates.
GPIO19 / GPIO20	Native USB D– / D+	Loses the USB link used for flashing and the REPL.
GPIO43 / GPIO44	UART0 console	Loses Thonny and every boot log — the tools you need when something else goes wrong.
GPIO0 , 3 , 45 , 46	Strapping pins	Decide boot mode at reset. A load on one can prevent normal boot intermittently, which is worse than never booting.
GPIO48	Onboard RGB LED	Usable, but you lose the LED. Revision-dependent.
GPIO47	<i>Damaged on this unit</i>	Physically damaged pad — this is why touch MISO is on GPIO39. Fine on a new board.

FREE PINS ON THIS BUILD

[GPIO1](#) [GPIO2](#) [GPIO6](#) [GPIO41](#) [GPIO42](#) are unclaimed by the firmware and safe to use. GPIO41 is the one already earmarked in the config comments for either the R307 WAKEUP line or the modem's NET LED — pick one, not both.

ADC NOTE

GPIO39 (touch MISO) is on ADC1. That is harmless here because it is used as a digital input, but if you ever add analogue sensing, remember ADC2 is unavailable while Wi-Fi is active on the ESP32 — a classic intermittent that only appears once the radio is up.

09 Bring-up procedure

Ordered so that the expensive mistakes are impossible before the cheap checks have passed. Do not skip ahead — debugging seven subsystems at once is not seven times harder, it is worse than that.

Stage 1 — before any power

- ☐ Continuity-check every ground back to the buck output terminal. One star point.
- ☐ Confirm the 3 A fuse is fitted in the +12 V feed.
- ☐ Confirm the 1N4007 is across the solenoid coil, cathode to the +12 V side.
- ☐ Confirm 470 μ F + 100 nF at the modem VIN and 100 μ F + 100 nF at the sensor.
- ☐ Confirm the modem antenna is fitted. Powering an LTE module without one can damage its PA.
- ☐ Confirm nothing on the TFT or touch header is connected to 5 V.

Stage 2 — set the rail

- ☐ Connect 12 V to the buck with **nothing on its output**.
- ☐ Meter the output terminals. Adjust the trimmer to 5.0 V.
- ☐ Power down before connecting anything.

USB AND 12 V AT THE SAME TIME

On the DevKitC-1 the USB VBUS and the 5V header pin sit on the same net with no ORing diode. **While flashing over USB, disconnect the 12 V supply** — or fit an SS34 Schottky in the buck's 5 V line so the two sources cannot fight. Two supplies back-feeding each other is how a bench session ends with a dead board.

Stage 3 — add loads one at a time

Power up, verify, power down, add the next. Each step is a few minutes.

1. **ESP32 alone.** It should enumerate over USB and give a REPL.
2. **Display.** `import diag_display` — walks the chain from a controller ID read up to a full-speed refresh.
3. **Touch.** `import calibrate`, tap the four crosshairs, paste the printed `TOUCH_CAL` block into `config.py`.
4. **Fingerprint sensor.** `import diag_fp` for raw bytes, or `import selftest` for the whole subsystem. Confirm the capacity reads 1000.
5. **Buzzer.** `import diag_buzzer` — plays the four cues, then sweeps.
6. **Relay, with no lock attached.** `import test_relay`. Listen for the click.
7. **Lock.** Reconnect the solenoid and repeat.
8. **Modem, last.** `import test_modem`.

Stage 4 — firmware and provisioning

1. Flash MicroPython **1.23 or newer**, build `ESP32_GENERIC_S3-SPIRAM_OCT`. Older builds lack `ssl.SSLContext`, which the HTTPS client needs.
2. On a laptop in the repo root, issue a device key:

```
node scripts/provision-device.mjs GYM-001 "Main Entrance" "Front door"
```

It prints the key **once**. Only its hash is stored server-side.

3. Copy `config.example.py` to `config.py`, fill in the Wi-Fi, the Supabase URL and anon key, and the device code and key from step 2.
4. Upload every `.py` file in `firmware/` to the device root. Upload `config.example.py` as `config.py`. Do not ship `calibrate.py` or `selftest.py` to a production device.
5. Run `import selftest`. Every line should read PASS. A `device_unauthorized` on the heartbeat means the key is wrong — go back to step 2.
6. Reset the board. `main.py` starts automatically.

Stage 5 — set the gym's Wi-Fi without a laptop

The terminal raises its own access point, `NOVA-SETUP-GYM-001`, WPA2-protected with a password derived from the device key. Join it from a phone and open `http://192.168.4.1/`. **The network name, its password and the address are all shown on the terminal's screen while the portal runs.** The form lists the networks in range, so nobody has to type an SSID on a phone keyboard.

The credential is proved before it is kept — the terminal connects with it while you are still standing there, and only writes it to flash once that succeeds. A typo fails at the form, not at the next reboot with nobody present. Up to four networks are remembered.

Two ways in: **Info** → **Device Health** → **Wi-Fi** on the terminal, or automatically at boot but *only* on a terminal that has never been given a network. The window is 300 s by design — a door left advertising a setup network is one anyone in range can point at their own router.

10 Fault-finding

Arranged by what you can see. In almost every case the first FAIL is the one to fix — everything below it usually fails for the same reason.

Diagnostic scripts on the device

RUN FROM THE REPL OVER USB

COMMAND	WHAT IT DOES
<code>import selftest</code>	Bring-up check, subsystem by subsystem. Start here.
<code>import diag_display</code>	Why the TFT is white: reads the controller ID, resets and inits by hand, fills primary colours, then re-runs at the real 40 MHz. Standalone, so a bug in the display driver cannot hide the fault.
<code>import diag_fp</code>	Why the sensor answers 0xFF. Sweeps baud rates and prints raw hex, driving the UART directly so the driver's framing cannot hide the bytes on the wire.
<code>import diag_buzzer</code>	Plays the four cues by name, then sweeps tones to reveal an active buzzer sold as passive.
<code>import diag_timing</code>	Loop timing.
<code>import test_relay</code>	Pulses the relay.
<code>import test_modem</code>	Climbs the ladder: modem answers → SIM ready → signal → LTE registration → data context with an IP → a real HTTPS heartbeat.
<code>import calibrate</code>	Prints the TOUCH_CAL numbers.

Also: **Info** → **Device Health** on the terminal itself runs the checks live rather than reporting cached status — it asks the sensor for its parameters and template count, stats the filesystem, and reads the network and server state. A banner says `ALL CHECKS PASS` or `n FAULTS`, and any failing row is red with the reason on it.

Power and boot

SYMPTOM	MOST LIKELY CAUSE	FIX
Nothing at all, no LEDs	Fuse, or buck output not set	Check the 3 A fuse, then meter the buck output for 5.0 V.
Board resets when the door unlocks	Missing flyback diode, or the lock sharing a ground path with logic	Fit the 1N4007 at the lock. Verify the star point.
Board resets during network activity	Modem inrush on a shared rail	Move modem VIN to the buck terminal and fit 470 µF + 100 nF there.
Boots intermittently	Something loading a strapping pin	See §8. Remove loads from GPIO0, 3, 45, 46.

Display

SYMPTOM	MOST LIKELY CAUSE	FIX
White screen	Backlight works, controller is not being reached	<code>diag_display</code> . A white panel means the controller is either not reset, not initialised, or not hearing SPI — from the front those look identical.

SYMPTOM	MOST LIKELY CAUSE	FIX
Display freezes when touch is used	Touch opened on hardware SPI(2)	Touch must stay on SoftSPI. Opening SPI(2) twice re-points the peripheral and freezes the panel with no error.
Visible tearing	Buses merged, or bus re-clocked per poll	Keep the two buses separate.
Touch reads offset or mirrored	Calibration or axis flags	<code>import calibrate</code> , then check <code>TOUCH_SWAP_XY</code> / <code>TOUCH_INVERT_X</code> / <code>TOUCH_INVERT_Y</code> .
Touch dead, panel fine	Non-touch panel variant	Check the header for T_ pins. No firmware can add a touch layer that is not there.

Fingerprint sensor

SYMPTOM	MOST LIKELY CAUSE	FIX
<code>Sensor ack failed: 0xFF</code>	A well-framed packet came back but the packet-id was not ACK. Three causes: baud mismatch, brown-out, or TX/RX wrong.	<code>diag_fp</code> tells them apart. Garbage at every rate is TX/RX; a truncated body is brown-out; a clean answer at another rate is baud.
<code>Sensor timeout</code> (OSError)	Nothing arriving at all	TX/RX swapped, no power, or no common ground.
Sensor starts a reply then sags mid-packet	Running on 3.3 V	The R307 needs 4.2–6 V. Move VCC to the buck's 5 V terminal. This is the classic fault after an R503 → R307 swap.
Blue capture light strobing constantly	Normal — that is the polling loop imaging	Not a fault. Fit the WAKEUP wire (\$6.1) to stop it and improve responsiveness.
"It only works sometimes"	Sensor blind between polls	Same fix: fit WAKEUP + VT.
Members above #200 never match	A 200-slot module sold as a 1000	<code>selftest</code> prints the real capacity from the chip. Replace the sensor.

4G modem

SYMPTOM	MOST LIKELY CAUSE	FIX
No reply to <code>AT</code> , PWR LED lit	One of exactly three things	TX/RX swapped; no common ground between buck and ESP32; or VDD/VREF left unconnected — the level shifter is referenced to it and without 3.3 V there nothing crosses in either direction even though every other wire is right.
No reply, PWR LED dark	VIN not on the buck, or the board is asleep	<code>test_modem.power_toggle()</code> pulses PWK. Many FS-MCore boards power up on their own, so a lit PWR LED means PWK is not the problem.
Garbage on the UART	Wrong baud rate	<code>test_modem.scan_baud()</code> .
Silence where <code>RDY</code> should be	Nothing arriving on GPIO7	<code>test_modem.listen()</code> — a healthy A7670C announces <code>RDY</code> , <code>+CPIN:</code> and <code>SMS DONE</code> a few seconds after power-up.

SYMPTOM	MOST LIKELY CAUSE	FIX
Answers AT, never registers	Brown-out on transmit	The signature power fault. VIN to the buck, 470 μ F at the connector.
+CPIN: SIM PIN	PIN-locked card	Not wiring. Disable the PIN on a phone first.
+CSQ of 99 that persists	Antenna	A 99 that clears in a few seconds is just the first scan.
CEREG 2 forever	Coverage, or a SIM with no data plan	Try the SIM in a phone.
IP reads 0.0.0.0	Wrong APN	Dialog dialogbb , Mobitel mobitel3g , Hutch hutch3g , Airtel airtelgprs.com .
TLS will not complete	SN1 or clock — see §6.3	test_modem.tls_probe() .

TESTING 4G CONCLUSIVELY

Set `LINK = "4g"` and **turn the Wi-Fi router off**. Under `"auto"` a working router means the modem is never reached, so a green run tells you nothing about it — the same trap as testing a generator without cutting the mains. The last rung of `test_modem` is a real heartbeat request, so passing it means the whole path works, not just that the radio attached.

Door and access

SYMPTOM	MOST LIKELY CAUSE	FIX
Relay clicks, lock does not move	Lock wiring or 12 V	Test the relay with the lock disconnected, then with it connected (§7).
No click at all	GPIO5, relay VCC, or polarity	<code>test_relay</code> . Confirm <code>RELAY_ACTIVE_HIGH = True</code> .
Lock never re-locks	Welded relay contacts	Replace the relay <i>and</i> fit the flyback diode, or it will happen again.
Member refused at the door but valid on the dashboard	Not a hardware fault	The door and the dashboard use the same server-side decision. An expired membership is refused identically in both. Check the member's payment period.
Footer shows <code>OFFLINE</code> and <code>Qn</code>	No uplink; events are queued	Normal degraded operation — the door still admits from the cached list. Restore the network and the queue drains. Event IDs are unique, so a replay cannot double-count.
Dashboard says the device is offline	3 minutes of silence	Heartbeat is every 60 s. Check the uplink.
<code>device_unauthorized</code>	Wrong or revoked device key	Re-run <code>provision-device.mjs</code> and update <code>config.py</code> .

11 Replacing a part

What is a straight swap and what needs configuration afterwards. Only one of these is a true drop-in.

PART	DROP-IN?	WHAT ELSE HAS TO HAPPEN
Relay module	Yes	Straight swap. Confirm the replacement is active-high, or set <code>RELAY_ACTIVE_HIGH</code> to match.
Buzzer	Yes	Must be <i>passive</i> . An active buzzer sounds at one fixed pitch and the four cues become indistinguishable.
Solenoid lock	Yes	Confirm fail-secure vs fail-safe (§7) and re-fit the flyback diode at the new lock.
R307 sensor	Almost	Wire per §6.1 — 5 V, not 3.3 V . Then run <code>selftest</code> and confirm the capacity reads 1000 before enrolling anyone. All existing fingerprint templates are lost — they live in the sensor, not on the server. Every member must re-enroll. Their membership, payments and attendance history are untouched.
TFT display	Almost	Must be the XPT2046 <i>touch</i> variant of the MD0671. Re-run <code>calibrate</code> afterwards — calibration is per-panel and the old numbers will be wrong.
A7670C modem	Almost	Move the SIM and antenna across. Set <code>MODEM_APN</code> if the operator changed. Verify with <code>test_modem</code> against the known-good figures in §6.3.
HW-688 buck	No	Set the output to 5.0 V with no load connected before wiring anything to it (§3).
ESP32-S3 board	No	Full re-provision — see below.

Replacing the controller

- Flash MicroPython 1.23+ `ESP32_GENERIC_S3-SPIRAM_OCT` .
- Issue a fresh device key: `node scripts/provision-device.mjs GYM-001 "Main Entrance"` . This **revokes the old key instantly**, which is exactly what you want if the old board is faulty or lost.
- Upload the firmware files and a `config.py` built from `config.example.py` .
- If the new board has an intact GPIO47 you may move touch MISO back to it — but then set `PIN_TCH_MISO = 47` . Leaving it on GPIO39 is equally fine.
- Re-run `calibrate` and `selftest` .
- Set the Wi-Fi through the setup portal (§9, stage 5).

Fingerprint templates live in the *sensor*, not the controller, so replacing the ESP32 alone does **not** lose enrollments.

WHERE ENROLLMENTS ACTUALLY LIVE

The template is stored in the R307's own memory. The server stores only the mapping from a member to a sensor slot number. So: **replace the controller, keep the fingerprints. Replace the sensor, lose them all**. If you are replacing a sensor, warn the gym before you start, because every member will need to re-enroll at the dashboard.

NEVER PASTE A DEVICE KEY INTO A CHAT, A SCREENSHOT OR THE REPO

`config.py` is gitignored for this reason. If a key is exposed, rotating it is one command and takes effect immediately.

12 Safety and compliance

EMERGENCY EGRESS MUST BE PHYSICAL HARDWARE

Nothing in this firmware should ever stand between a person and the way out. Getting *out* of the building must not depend on the ESP32, the network, the sensor or the software — it must be a mechanical handle, a push bar, or a break-glass unit wired to cut power to the lock directly. A locked door with a crashed controller is a fire exit that does not open.

This is a legal requirement in most jurisdictions, not a design preference. Have the egress arrangement checked against local fire regulations before the terminal goes into service.

POWER LOSS BEHAVIOUR

With a fail-secure lock, a power cut leaves the door **locked**. That is the correct default for security, and it is the reason the egress hardware above is non-negotiable. Make sure the gym's staff know what happens to the door in a power cut, and that the manual release is unobstructed.

Electrical

- Only the 12 V PSU touches mains. Everything in this document is extra-low voltage. Any mains work is for a qualified electrician.
- The 3 A fuse goes in the +12 V feed before anything else, and is the first thing to check when the terminal is completely dead.
- Route the 12 V lock wiring away from the SPI ribbon to the display. Switching an inductive load next to a 40 MHz bus is asking for corrupted frames even with the diode fitted.
- Enclose the buck and the relay contacts. They are the only places with exposed 12 V.

Data and privacy

- The sensor stores mathematical templates, not images, and a template cannot be reversed into a fingerprint. Templates never leave the sensor — the offline cache holds only `{slot, name, allowed}`, no biometric data.
- The device holds only the anon key and its own device key. Neither can read the members table.
- Expiry denies access but never deletes a template or attendance history. Removing a fingerprint clears the mapping only.

ONE OUTSTANDING ITEM BEFORE PUBLIC DEPLOYMENT

TLS certificate verification is currently **off** (`VERIFY_TLS = False`), which is the MicroPython norm but does mean the connection is encrypted without proving who is on the other end — someone on the same network could impersonate the server. To close it: download the ISRG Root X1 CA in DER form, upload it as `isrg_root_x1.der`, and set `VERIFY_TLS = True`. Do this before the terminal goes on a public gym network. The certificate expires in 2035, so diarise a re-upload.

Routine maintenance

INTERVAL	TASK
Monthly	Wipe the sensor platen with a dry microfibre cloth. Most "the sensor got worse" reports are a dirty platen.
Quarterly	Open Info → Device Health and confirm <code>ALL CHECKS PASS</code> . Note the free template slots — it warns amber below 5%.
Quarterly	Check the door lock alignment and that the manual egress release is unobstructed.

INTERVAL	TASK
Annually	Re-torque the screw terminals on the buck and relay. Thermal cycling loosens them.
Before 2035	Re-upload the root CA certificate if TLS verification has been enabled.

Where to look next

FILE IN THE REPOSITORY	CONTENTS
firmware/config.py	The authoritative pin map. Every number in §4 comes from here.
firmware/README.md	Firmware detail, the modem investigation notes, tuning parameters.
docs/API.md	The server endpoints the device calls.
docs/DEPLOYMENT.md	Backend and dashboard deployment.
docs/SECURITY.md	Security checklist.
docs/HARDWARE_WIRING.html	Obsolete. Superseded by this document — its pin map does not match the running firmware.